

Question 3 Report

To choose a learning rate for Algorithm 1, I evaluated the test loss over a range of values of t (0.0625, 0.125, 0.25, 0.5, 1, 2, 4, 8, 16). The resulting convergence plots are seen in Figure 1. For small learning rates, such as $t = 0.0625$ and $t = 0.125$, the test loss decreases smoothly and monotonically towards zero as the iterations increase, indicating stable convergence. At $t = 0.25$, the loss still decreases overall but begins to oscillate, showing that the method is close to the limit of stability. For larger learning rates ($t \geq 0.5$), the oscillations become more pronounced. At $t = 2, 4, 8$ and 16 , the loss oscillates violently, with spikes that diverge out of the visible range of the plots. From these observations, $t = 0.125$ is the largest learning rate to produce stable convergence, so I therefore used it when applying Algorithm 1 to the X-ray spectra.

With $t = 0.125$, Algorithm 1 converges quickly and without significant oscillation. The predicted classification image obtained from θ_{s_star} consists of two large, coherent regions with a clear boundary. This is consistent with what can be expected from undersampling features, since they only capture every 100th measurement and therefore compress the spectra into a broad, low-resolution representation. The resulting classification image suggests that the model has learned meaningful distinctions between the spectral patterns, and the predicted labels look accurate.

Question 5 Report

In Question 5, I replaced the undersampling features from Question 3 with clustering features generated by `ClusterFeaturesFit` and `ClusterFeaturesPredict`. To choose an appropriate number of clusters, I used the elbow method by computing the KMeans inertia for $K = 1$ to $K = 10$. As shown in Figure 2, the inertia drops sharply between $K = 2$ and $K = 3$ before flattening out completely, which indicates diminishing benefit from increasing K . Therefore, I used $K = 3$ when constructing the clustering features.

To understand how informative the clustering features were, I plotted the clustering features for each cluster. Figure 3 shows clear separation between the three groups, although Cluster 0 and Cluster 2 have very low variance, indicating that only one of the features is strongly informative, which limits how well the classes can be separated by a linear softmax model.

To choose a learning rate for Algorithm 1, I evaluated the test loss over a range of values of t (0.0625, 0.125, 0.25, 0.5, 1, 2, 4, 8, 16). The resulting convergence plots are seen in Figure 4. For small learning rates, such as $t = 0.0625$, $t = 0.125$ and $t = 0.25$, the test loss decreases smoothly and monotonically towards zero as the iterations increase, indicating stable convergence. At $t = 0.5$ and $t = 1$, the loss still decreases overall but begins to oscillate, showing that the method is close to the limit of stability. For $t = 2$, the oscillations become more pronounced. At $t = 4, 8$ and 16 , the loss oscillates violently, with spikes that diverge out of the visible range of the plots.

When trying to form a classification image with $t = 0.0625$ or $t = 0.125$, the model cannot meaningfully distinguish between different spectral patterns and so produces a degenerate classification image. This may be due to the convergence being too slow or becoming stuck in a flat region of the loss function. Therefore, I used $t = 0.25$ when applying Algorithm 1 to the X-ray spectra.

Using $K = 3$ and $t = 0.25$, Algorithm 1 converged smoothly and without significant oscillation, producing a clear two-region classification image. As mentioned above, only one of the clustering features is informative, so the classifier can only form a small number of decision boundaries, which is why the resulting classification image only has two broad classes in relatively simple, broad regions. As a result, the predicted labels look accurate and are consistent with the underlying data.

In Question 3, the stable learning rate region was much smaller, with values above $t = 0.125$ causing strong oscillations or divergence. This was likely because of the features having higher variance, so the gradients were larger. In contrast, in Question 5, the stable learning rate was much higher, with values above $t = 0.25$ causing strong oscillations or divergence. As opposed to Question 3, in Question 5, the features have much lower variance, so the gradients are smaller, and so the optimisation can tolerate larger learning rates. Moreover, the Question 5 classification image is much more fragmented than the Question 3 classification image as a result of the lower variance, however, both methods produce consistent two-class behaviour.

The model could be improved in several ways. Standardising the clustering features would prevent the informative feature from dominating the optimisation and increase the other two features' effective variability. Alternatively, combining clustering features with the undersampling features from Question 3 would provide a more informative representation of the spectra and likely lead to smoother and more accurate classifications.

Question 6

$$Q6) \quad L_0(\theta) = -\frac{1}{N} \sum_{i=0}^{N-1} \sum_{k=0}^{M-1} y_{i,k} \log h_{\theta_k}(\hat{x}_i), \text{ where } h_{\theta_k}(\hat{x}_i) = \frac{\exp(\langle \hat{x}_i, \theta_k \rangle)}{\sum_{k=0}^{M-1} \exp(\langle \hat{x}_i, \theta_k \rangle)}$$

$$x_i = \begin{bmatrix} 1 \\ \hat{x}_i \end{bmatrix} \in \mathbb{R}^{1 \times (n+1)}$$

taking the partial derivative of $L_0(\theta)$ with respect to one element $\theta_{j,k}$

$$\frac{\partial L_0}{\partial \theta_{j,k}} = -\frac{1}{N} \sum_{i=0}^{N-1} y_{i,k} \underbrace{\frac{\partial}{\partial \theta_{j,k}} \log h_{\theta_k}(\hat{x}_i)}_{(1)}$$

deriving (1), let $z_{i,r} = \langle x_i, \theta_r \rangle$, $h_{i,r} = h_{\theta_r}(\hat{x}_i)$

$$\begin{aligned} \text{we } h_{i,r} &= \frac{e^{z_{i,r}}}{\sum_{s=0}^{M-1} e^{z_{i,s}}} \\ \therefore \frac{\partial h_{i,r}}{\partial z_{i,k}} &= \frac{(e^{z_{i,r}})' \sum_{s=0}^{M-1} e^{z_{i,s}} - e^{z_{i,r}} (\sum_{s=0}^{M-1} e^{z_{i,s}})'}{(\sum_{s=0}^{M-1} e^{z_{i,s}})^2} \\ &= \frac{\delta_{rk} e^{z_{i,k}} \sum_{s=0}^{M-1} e^{z_{i,s}} - e^{z_{i,r}} e^{z_{i,k}}}{(\sum_{s=0}^{M-1} e^{z_{i,s}})^2}, \text{ and that } \delta_{rk} = \begin{cases} 1, & \text{if } r=k \\ 0, & \text{if } r \neq k \end{cases} \\ &= \frac{e^{z_{i,r}}}{\sum_{s=0}^{M-1} e^{z_{i,s}}} \cdot \frac{\delta_{rk} \sum_{s=0}^{M-1} e^{z_{i,s}} - e^{z_{i,k}}}{\sum_{s=0}^{M-1} e^{z_{i,s}}} \\ &= \frac{e^{z_{i,r}}}{\sum_{s=0}^{M-1} e^{z_{i,s}}} \cdot (\delta_{rk} - h_{i,k}) \\ &= h_{i,r} (\delta_{rk} - h_{i,k}) \end{aligned}$$

$$\begin{aligned} \therefore \frac{\partial}{\partial \theta_{j,k}} \log h_{\theta_k}(\hat{x}_i) &= \frac{1}{h_{\theta_k}(\hat{x}_i)} \cdot h_{\theta_k}(\hat{x}_i) (1 - h_{\theta_k}(\hat{x}_i)) x_{i,j} \\ &= (1 - h_{\theta_k}(\hat{x}_i)) x_{i,j} \quad \text{--- (2)} \end{aligned}$$

substituting ② into $\frac{\partial L_0}{\partial \theta_{j,k}}$,

$$\frac{\partial L_0}{\partial \theta_{j,k}} = -\frac{1}{N} \sum_{i=0}^{N-1} y_{i,k} (1 - h_{\theta_k}(\hat{x}_i)) x_{i,j}$$

$$= \frac{1}{N} \sum_{i=0}^{N-1} (h_{\theta_k}(\hat{x}_i) - y_{i,k}) x_{i,j}$$

$$\therefore \nabla L_0(\theta) = \frac{1}{N} \sum_{i=0}^{N-1} x_i^T (h_{\theta}(\hat{x}_i) - y_i) \quad x_i = [\hat{x}_i] \text{ when in matrix form, as seen in (8)}$$

Question 7

Q7) Lemma 4.25 states that if L is β -smooth, then $L(\theta^{k+1}) \leq L(\theta^k) - t_k(1 - t_k \frac{\beta}{2}) \|\nabla L(\theta^k)\|^2$, where t_k is the learning rate at iteration k .

$$\text{let } D(t) = t(1 - t \frac{\beta}{2}) \\ = t - \frac{\beta}{2} t^2$$

since $D(t)$ is the loss decrease factor, by maximising it, we maximise the possible decrease in loss per iteration, which then gives the maximum drop in loss and fastest convergence. Consequently this gives optimal learning rate t . Therefore,

$$D'(t) = 1 - \beta t$$

$$\text{setting } D'(t) \text{ to } 0, \quad 1 - \beta t = 0 \\ t = \frac{1}{\beta}$$

$\therefore$ the optimal learning rate in Algorithm 1 is $t = \frac{1}{\beta}$

AGD can be advantageous over the standard Gradient Descent method when the optimisation problem is poorly scaled, meaning that the progress is slow in some directions and unstable in others. In these cases, the standard Gradient Descent tends to oscillate across steep directions of the loss surface, resulting in slow progress along flatter directions. Algorithm 1 rescales the gradient using an estimate of its squared magnitude, which dampens large gradients (preventing overshooting) and amplifies small gradients (allowing for faster progress). As a result, AGD produces faster and more stable convergence than the standard Gradient Descent method.